

Lab 05 — Antwoorden met toelichting

Elk antwoord volgt hetzelfde schema: het antwoord, het mechanisme, de nuance of valkuil, een voorbeeld dat je aan de terminal kunt nagaan.

Vraag 1 — 950 MB geweigerd door de beveiliging

Antwoord. (1) **De broncode zit in de image:** wie de image pullt, leest de applicatie — en vaak lokale configuratiebestanden. (2) **Het aanvalsoppervlak:** JDK, Maven, `git`, `curl`, een volledige shell, honderden Debian-pakketten — evenveel tools voor een aanvaller die code kan uitvoeren, en een scanrapport met 300 CVE's dat niemand zal behandelen. (3) **De repository** `~/m2` bevat de gedownloade artefacten en, vaak, een `settings.xml` met de credentials van de private Maven-repository. Het moeilijkst te verhelpen zonder multi-stage is (2): je kunt de bronnen en `.m2` met `rm` verwijderen (slecht: de lagen bewaren ze, lab 02), maar je kunt de JDK niet weghalen uit een image die van een JDK-image vertrekt.

Waarom. De uiteindelijke image is de laatste `FROM` plus wat je eraan toevoegt; je kunt de basisimage niet "afrekken". Alleen een tweede `FROM` op een minimale basis, met `COPY --from`, verandert van basis.

Nuance. De grootte zelf heeft een operationele kost (pulltijd bij incident, opslag in de registry), maar het is het zwakste argument tegenover een veiligheidsverantwoordelijke — de inhoud telt meer dan het gewicht.

Voorbeeld.

```
podman run --rm --entrypoint sh api-mono:1.0 -c 'ls /app; javac -version; ls ~/m2 2>/dev/null'
```

Vraag 2 — Drie `FROM`s

Antwoord. De **laatste** `FROM` levert de uiteindelijke image; de twee andere zijn tijdelijke omgevingen, vernietigd op het einde van de build (alleen hun cache blijft). Als geen enkele `COPY --from` (noch een latere `FROM`) naar de tweede stage verwijst, **bouwt Buildah ze helemaal niet:** ze wordt genegeerd. Je ziet het in de uitvoer: de stages zijn genummerd `[1/3]`, `[2/3]`, `[3/3]`, en het nummer van de nutteloze stage verschijnt nooit.

Waarom. De engine berekent eerst de afhankelijkheidsgraaf tussen de stages uit de `--from`s, en bouwt dan alleen wat naar het doel leidt (de laatste stage, of `--target`).

Nuance. BuildKit doet hetzelfde en bouwt bovendien onafhankelijke stages parallel. Een "nutteloze" stage heeft een legitiem gebruik: een teststage (`RUN mvn test`) die men alleen bouwt met `--target test` in de CI.

Voorbeeld.

```
podman build -f Dockerfile.unused -t u . 2>&1 | grep STEP      # alleen [2/3] en [3/3], nooit [1/3]
```

Vraag 3 — 420 MB en de bronnen

Antwoord. `COPY --from=build /app /app` kopieert **de hele werkmap** van de stage `build`: `Api.java` / `src`, `pom.xml`, `target/` met zijn klassen, en de JAR. Correctie: alleen het artefact

kopiëren.

```
COPY --from=build /app/target/api.jar /app/api.jar
```

(en de `ENTRYPOINT` aanpassen naar `/app/api.jar .`)

Waarom. Multi-stage filtert op zichzelf niets: het zet in de uiteindelijke image alleen wat `COPY --from` vraagt. Een map vragen is de hele inhoud ervan vragen.

Nuance. Bevat de image, eens gecorrigeerd, nog een `.m2`? Nee — `.m2` zit in `/root` van de build-stage, niet in `/app`. Maar dat is geluk, geen garantie: de regel is **benoemde bestanden** kopiëren.

Voorbeeld.

```
podman run --rm --entrypoint ls api-multi:1.0 /src      # No such file or directory: goed teken
```

Vraag 4 — `ng serve` in productie

Antwoord. Vier redenen: (1) `ng serve` is een **ontwikkelserver** — niet geoptimaliseerd, zonder compressie of HTTP-cache, expliciet gedocumenteerd als niet bedoeld voor productie; (2) de image bevat **Node, de bronnen en `node_modules`** (vaak > 1 GB): aanvalsoppervlak en codelek; (3) de build gebeurt niet **één keer** maar bij elke start, in "watch"-modus, met *source maps* geactiveerd; (4) geen scheiding tussen de applicatie en haar tooling — een kwetsbare ontwikkelafhankelijkheid zit in productie. In de plaats: `ng build --configuration production` in een Node-stage, dan `COPY --from` van de map `dist/.../browser` naar een image `nginx:alpine` met een nginx-configuratie die `index.html` teruggeeft voor de Angular-routes.

Waarom. Een gecompileerde Angular-frontend is statisch. Ze serveren vraagt alleen een bestandsserver; al de rest is build, die bij de weggegooid stage hoort.

Nuance. Er is één geval waarin Node in productie blijft: server-side rendering (Angular SSR / Universal). Dat is dan een **andere** applicatie, met een eigen Dockerfile, en nog altijd geen `ng serve`.

Voorbeeld.

```
podman images --format '{{.Repository}} {{.Size}}' | grep -E 'web-multi|node'      # 64 MB
tegenover 167 MB (zonder node_modules!)
```

Vraag 5 — `UnsatisfiableLinkError` na Alpine

Antwoord. De PDF-generatie steunt hoogstwaarschijnlijk op een **native bibliotheek** (`.so`) gecompileerd voor `glibc` — lettertypen, rendering, compressie. Alpine levert `musl`: de loader weigert de bibliotheek, Java gooit `UnsatisfiableLinkError`. Het commando dat het zou hebben getoond: `ldd --version` in elke image (`musl libc` tegenover `GLIBC`). De migratie had getest moeten worden met de **echte verwerkingen** (niet alleen `/actuator/health`), op een acceptatieomgeving, met een terugkeerplan — en de beslissing component per component genomen.

Waarom. Een native bibliotheek is gebonden aan een precieze `libc`; het is geen portable bytecode. Twee weken vertraging, omdat de PDF-generatie misschien alleen op het einde van de maand draait.

Nuance. Er bestaan alternatieven: de image `eclipse-temurin:21-jre-ubi9-minimal` (Red Hat, `glibc`, ~100 MB), of `gcompat` installeren op Alpine (broos). En het is niet eigen aan Java: Python, Node met native modules, hebben precies dezelfde valkuil.

Voorbeeld.

```
podman run --rm --entrypoint sh docker.io/library/eclipse-temurin:21-jre-alpine -c 'ldd --version 2>&1 | head -1' # musl libc
podman run --rm --entrypoint sh docker.io/library/eclipse-temurin:21-jre -c 'ldd --version | head -1' # GLIBC 2.xx
```

Vraag 6 — Multi-stage en geheimen

Antwoord. Een `rm` maakt een laag aan die het bestand verbergt; de laag die het schreef, blijft in de image (lab 02). Een weggegooide stage daarentegen zit **niet** in de uiteindelijke image: geen enkele van haar lagen staat erin. Als het geheim alleen in een weggegooide stage geschreven werd, bestaat het nergens in het gepubliceerde artefact. Multi-stage beschermt niet als het geheim naar de laatste stage **gekopieerd** wordt (`COPY --from=build /app` met het geheim erin), of als het artefact zelf het geabsorbeerd heeft (een `application.yml` met wachtwoord verpakt in de JAR), of als het geheim via een `ARG` van de laatste stage passeert (zichtbaar in `history`).

Waarom. Uiteindelijke image = lagen van de laatste `FROM` + lagen geproduceerd door zijn instructies. Een vorige stage draagt alleen bij wat een `COPY --from` eruit haalt.

Nuance. De moderne oplossing is `RUN --mount=type=secret`: het geheim is beschikbaar tijdens één enkele instructie, in eender welke stage, zonder ooit een laag te worden. Multi-stage blijft de structurele garantie, de *secret mount* de garantie per instructie.

Voorbeeld.

```
podman build --secret id=pw,src=pw.txt -f Dockerfile.secret -t sec .
podman run --rm sec ls /run/secrets # No such file or directory
```

Vraag 7 — JAR in één blok tegenover lagen

Antwoord. (a) Eén laag van 50 MB die bij elke build verandert: de `push` en elke `pull` dragen **50 MB** over. (b) Vier lagen: afhankelijkheden (~45 MB, ongewijzigd), loader (~1 MB, ongewijzigd), snapshots (0), applicatie (~5 MB): de uitrol draagt **~5 MB** over. Verhouding 10. (a) blijft aanvaardbaar omdat 50 MB op een datacenternetwerk één seconde kost, omdat de JRE (180 MB) toch gedeeld wordt, en omdat (b) een stage, een andere `ENTRYPOINT` (`org.springframework.boot.loader...` of `java -jar` op de map) en uit te leggen complexiteit toevoegt.

Waarom. De overdracht is differentieel per laag; wat telt, is de grootte van de laag die verandert, niet die van de image.

Nuance. (b) wordt rendabel wanneer men vaak uitrolt naar veel nodes, of over een traag netwerk (edge, verre sites). En het principe geldt zonder Spring: `lib/` (stabiel) en `classes/` (vluchtig) scheiden volstaat.

Voorbeeld.

```
podman history api:1.0 --format 'table {{.Size}}\t{{.CreatedBy}}' | head -6 # één laag van 50 MB, of vier lagen
```

Vraag 8 — 90 seconden die 7 minuten werden

Antwoord. De nieuwe agent heeft een **lege cache**: de buildcache (lagen) leeft op de machine die bouwt, en een nieuwe agent — of een kortstondige agent die bij elke pipeline opnieuw wordt aangemaakt — vertrekt van nul. De 5 minuten van `dependency:go-offline` worden dus opnieuw betaald. Twee mechanismen: (1) een **cache mount** (`RUN --mount=type=cache,target=/root/.m2`), die de Maven-repository op de agent tussen builds bewaart, zelfs als `pom.xml` verandert; (2) een **externe cache** — `--cache-from` / `--cache-to` naar de registry — waarmee een nieuwe agent de lagen van een vorige build kan ophalen. Met rootless Podman zit de cache (lagen en *cache mounts*) in `~/.local/share/containers/storage` van de gebruiker die bouwt: een agent die elke job onder een andere gebruiker of `home` uitvoert, of die zijn `home` vernietigt, heeft nooit een cache.

Waarom. "Er is niets veranderd" is waar aan de kant van de bronnen, onwaar aan de kant van de cache: de cache is een lokale toestand van de machine, geen eigenschap van de Dockerfile.

Nuance. Kortstondige agents zijn gewild (isolatie, reproduceerbaarheid); het antwoord is de cache **expliciet en extern** maken, niet agents lang in leven houden. En een `--no-cache` in de pipeline "om zeker te zijn" veroorzaakt precies dit symptoom, permanent.

Voorbeeld.

```
podman build --cache-to registry.intern/mijnapp/api-cache --cache-from
registry.intern/mijnapp/api-cache -t api:1.5 .
podman info --format '{{.Store.GraphRoot}}' # waar de cache van deze gebruiker leeft
```

Vraag 9 — Wat je verliest met distroless

Antwoord. (1) `podman exec -it ... sh`: geen shell, dus geen interactieve verkenning, geen `cat` van een configuratiebestand, geen `curl localhost:8080/actuator`. Compensatie: blootgestelde observeerbaarheidsendpoints (`/actuator/health`, `/info`, `/env`), gestructureerde en volledige logs op `stdout`, en `podman cp` om een bestand uit te halen. (2) **De diagnosetools** (`jcmd`, `jstack`, `ps`, `netstat`): niets om een *thread dump* te nemen of de sockets te zien. Compensatie: een debug-*sidecar*-container die de namespaces deelt (`podman run --pid=container:api --network=container:api debug-image`), of JMX/Actuator-tools (`/actuator/threaddump`) blootgesteld op het interne netwerk.

Waarom. Alles wat de beheerder gebruikt om in een container te geraken, gebruikt een aanvaller ook. Distroless schrapt beide tegelijk; de observeerbaarheid moet dus **buiten** de image verhuizen.

Nuance. Distroless images bestaan in een `:debug`-variant met een busybox-shell — nuttig in acceptatie, verboden in productie. En Kubernetes biedt `kubectld debug` met kortstondige containers voor precies die behoefte.

Voorbeeld.

```
podman exec d sh -c ls # executable file `sh` not found
curl -s localhost:18082/actuator/health # de observeerbaarheid verloopt via HTTP
```

Vraag 10 — De cache mount, VOLUME, en # syntax=

Antwoord. De gegevens zitten in een **cachemap beheerd door de build-engine** (BuildKit of Buildah), op de machine die bouwt — bij rootless Podman in je gebruikersopslag. Ze wordt in de buildcontainer gemount **alleen tijdens de instructie**, en dan ontkoppeld: niets wordt in een laag geschreven, dus niets in de image. Een **VOLUME** is het omgekeerde: een verklaring in de image die, bij de **uitvoering**, een volume voor de container aanmaakt; ze heeft geen enkel effect tijdens de build. Zonder `# syntax=docker/dockerfile:1`: onder Docker (recente versies, BuildKit standaard) werkt `--mount` hoe dan ook met de huidige stabiele syntax; de regel diende alleen om een recentere frontend-versie af te dwingen. Onder Podman wordt de regel **genegeerd** (Buildah heeft geen frontend) en werkt `--mount` van nature.

Waarom. De cache mount is een mechanisme van de build-engine; de **VOLUME** een mechanisme van de runtime. Ze delen alleen het woord "mount".

Nuance. De cache mount wordt niet gedeeld tussen machines of gebruikers, en hij wordt niet ongeldig gemaakt door de inhoud: een corrupte Maven-repository blijft erin. `podman system prune --build-cache` ... bestaat nog niet: je verwijdert de opslag of gebruikt `id=` om van cache te wisselen.

Voorbeeld.

```
podman build --no-cache -f Dockerfile.cache -t c . 2>&1 | grep dep-      # de markers stapelen
zich op van build tot build
podman run --rm c ls /root/.m2                                         # afwezig in de image
```

Vraag 11 — "Multi-stage dient tot niets voor Angular"

Antwoord. Zonder multi-stage is de uiteindelijke image de image waarin `ng build` draaide: `node:22-alpine` (~170 MB) **plus** `node_modules` (500 MB tot 1 GB) **plus** de TypeScript-bronnen **plus** de `dist/` — en je moet er nog een server aan toevoegen om `dist/` te serveren. Met multi-stage: `nginx:alpine` (64 MB) plus enkele MB statische bestanden. Het verschil is een factor 10 tot 20, en de inhoud verandert van aard: geen Node meer, geen bronnen, geen buildafhankelijkheden.

Waarom. Dat het *resultaat* statisch is, is precies het argument **voor** multi-stage: aangezien de uitvoering niets nodig heeft van wat diende om te bouwen, hou je beter niets bij.

Nuance. Zonder container kan een team ook `ng build` in de CI doen en `dist/` in één stap naar een nginx-image kopiëren (`COPY dist/ /usr/share/nginx/html`). Dat is een "multi-stage" waarvan de eerste stage de CI is — geldig, maar de build is niet meer reproduceerbaar vanuit de Dockerfile alleen.

Voorbeeld.

```
podman images --format '{{.Repository}} {{.Size}}' | grep -E 'web-multi|node'      # 64.2 MB
tegenover 167 MB
```

Vraag 12 — /app/dist: no such file or directory

Antwoord. De stage `build` heeft `WORKDIR /src`: de build levert `/src/dist`, niet `/app/dist`. Correctie: `COPY --from=build /src/dist/<project>/browser /usr/share/nginx/html` (de submap

hangt af van de Angular-versie en de projectnaam). Om de diagnose te stellen zonder te gokken: `podman build --target build -t dbg .` en dan `podman run --rm dbg find / -name index.html -path '*dist*'.`

Waarom. `COPY --from` kopieert vanuit het bestandssysteem van de stage, met **absolute** paden van die stage. Een fout in `WORKDIR` of in de structuur van `dist/` is onzichtbaar zolang men er niet in kijkt.

Nuance. Sinds Angular 17 is de standaarduitvoer `dist/<project>/browser/`; daarvoor `dist/<project>/`. `--target` vermijdt dat je op je geheugen moet vertrouwen.

Voorbeeld.

```
podman build --target build -t dbg . && podman run --rm dbg ls -R /src/dist | head
```

Vraag 13 — `RUN mvn test` in de Dockerfile

Antwoord. In de build-stage, **na** de compilatie en **vóór** de `package` (of in één enkele `mvn package` zonder `-DskipTests`): als een test faalt, faalt `RUN`, stopt de build, en wordt er geen image geproduceerd. Nadeel: de tests draaien in een geïsoleerde buildcontainer — geen JUnit-rapport bruikbaar door de CI (het zit in een weggegooide stage, tenzij je het kopieert met `--target` of `--output`), geen gemakkelijk bereikbare testdatabase (Testcontainers heeft een engine nodig), en de buildtijd van de image omvat die van de tests, zelfs als men alleen wilde herbouwen.

Waarom. De Dockerfile is een goede garant ("geen image zonder groene tests") maar een slechte rapporteringstool.

Nuance. Het gangbare compromis: de CI voert de tests **en** de build van de image in twee jobs uit, met de build afhankelijk van het slagen van de tests; de Dockerfile houdt `-DskipTests` om snel te blijven. Je krijgt het rapport en de garantie, tegen de prijs van een afhankelijkheid van de CI.

Voorbeeld.

```
RUN mvn -q test # rood -> de build stopt hier
RUN mvn -q package -DskipTests
```

Vraag 14 — Eén laag van 250 MB of vijf lagen van 280 MB

Antwoord. De image van **280 MB in vijf lagen** wordt sneller uitgerold bij een update van de code: alleen de vluchtige laag (enkele MB) wordt overgedragen, de vier andere staan al op de nodes en in de registry. De image van 250 MB in één laag draagt bij elke versie 250 MB opnieuw over. Het antwoord keert om wanneer de nodes **niets** hebben (eerste uitrol, nieuwe node, geleegde registry, of een tagstrategie die elke keer alles verandert): dan is $250 < 280$, en wint de enkele laag — nipt.

Waarom. De overdrachtskost is die van de ontbrekende lagen, niet van de image. De stabiliteit van de lagen is meer waard dan hun aantal.

Nuance. `--squash` (Buildah) of een minimale basis kunnen de 280 MB naar 250 brengen zonder de lagen te verliezen: de twee criteria sluiten elkaar niet uit. En de winst bestaat alleen als de stabiele lagen **bit voor bit identiek** zijn van de ene build tot de andere — reproduceerbaarheid van de build vereist (geen niet-vastgepinde `apt-get update` in een "stabiele" laag).

Voorbeeld.

```
podman push registry.intern/mijnapp/api:1.5.1    # stabiele blobs: onmiddellijk; alleen de  
codelaag wordt gekopieerd
```